Système Anti-Triche pour Challenges de Code

Détection automatique du code généré par IA,
du plagiat et du copier-coller

API REST déployée sur Hugging Face Spaces

Apprentissage automatique continu
Le modèle devient plus intelligent avec le temps

URL de l'API : https://negzaoui-anti-cheat-coding.hf.space

GitHub : https://github.com/negzaoui16/anti-cheat-coding

Technologie : Python, FastAPI, scikit-learn

Déploiement : Docker, Hugging Face Spaces

2 février 2026

Table des matières

1 Introduction **3**
- 1.1 Contexte . . . 3
- 1.2 Objectif du Système . . . 3
- 1.3 Caractéristiques Clés . . . 4

2 Architecture du Système **5**
- 2.1 Vue d'Ensemble . . . 5
- 2.2 Structure des Fichiers . . . 5
- 2.3 Description des Modules . . . 6
  - 2.3.1 Module de Détection IA (model_ia.py) . . . 6
  - 2.3.2 Module de Détection Plagiat (plagiarism.py) . . . 6
  - 2.3.3 Module Copier-Coller (copy_paste.py) . . . 6
  - 2.3.4 Module de Scoring (scoring.py) . . . 6

3 API REST **8**
- 3.1 URL de Base . . . 8
- 3.2 Endpoints Disponibles . . . 8
- 3.3 Endpoint POST /analyze . . . 8
  - 3.3.1 Requête . . . 8
  - 3.3.2 Réponse . . . 9
- 3.4 Endpoint POST /analyze_github . . . 9
  - 3.4.1 Requête . . . 9
  - 3.4.2 Réponse . . . 9

4 Apprentissage Automatique **10**
- 4.1 Fonctionnement . . . 10
- 4.2 Seuils d'Auto-Labellisation . . . 11
- 4.3 Configuration . . . 11
- 4.4 Intervention Humaine (Optionnelle) . . . 11

5 Résultats des Tests **12**
- 5.1 Validation Complète . . . 12
- 5.2 Tests Effectués . . . 12
- 5.3 Test des 12 Utilisateurs . . . 13
- 5.4 Test GitHub . . . 13

6 Code Source Principal **14**
6.1 API REST (api.py) 14
6.2 Extraction de Features (features.py) 14
6.3 Scoring (scoring.py) 15

7 Conclusion **16**
7.1 Récapitulatif 16
7.2 Points Forts 16
7.3 Utilisation Recommandée 16

Chapitre 1

Introduction

1.1 Contexte

Dans le cadre des plateformes de challenges de code (coding challenges), il est essentiel de détecter les comportements frauduleux tels que :

1. **Code généré par IA** : ChatGPT, GitHub Copilot, Claude, etc.
2. **Plagiat** : Code copié d'autres participants
3. **Copier-coller** : Code collé depuis internet ou d'autres sources

1.2 Objectif du Système

Le système anti-triche a pour objectif de :

Objectifs principaux

1. **Analyser** le code soumis par les participants
2. **Calculer** un score de suspicion (0 à 1)
3. **Retourner** un verdict : OK, SUSPECT, ou TRICHE PROBABLE
4. **Apprendre** automatiquement et devenir plus intelligent avec le temps
5. **Ne pas bloquer** automatiquement (décision humaine possible)

1.3 Caractéristiques Clés

Caractéristique	Description
Apprentissage Automatique	Le modèle apprend tout seul
API REST	Accessible depuis n'importe quelle application
Déploiement Cloud	Hugging Face Spaces (toujours en ligne)
CPU uniquement	Pas besoin de GPU
Open Source	Code disponible sur GitHub

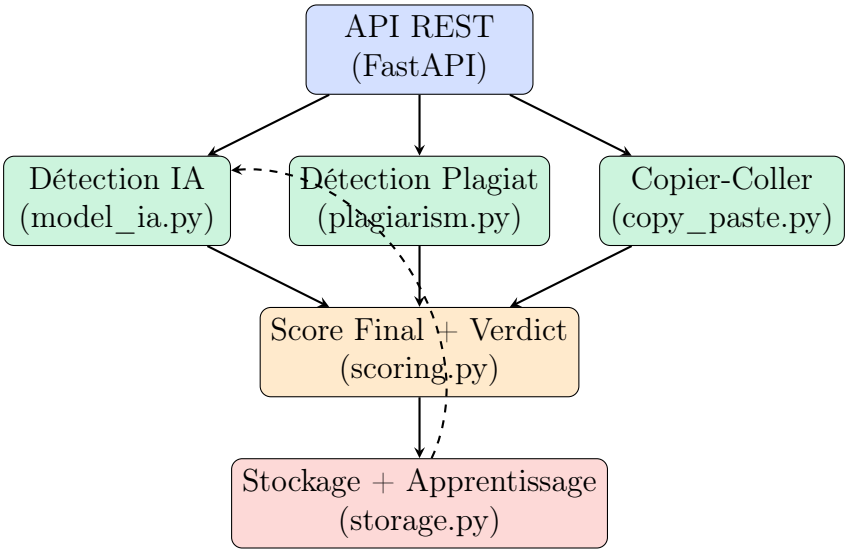
TABLE 1.1 – Caractéristiques principales du système

Chapitre 2

Architecture du Système

2.1 Vue d'Ensemble

Le système est composé de plusieurs modules interconnectés :



2.2 Structure des Fichiers

Listing 2.1 – Arborescence du projet

```
Modele_Cheat/
|-- config.py                  # Configuration globale
|-- requirements.txt           # Dependances Python
|-- Dockerfile                 # Deploiement Docker
|-- src/
|   |-- api.py                # API REST FastAPI
|   |-- model_ia.py           # Modele ML (RandomForest)
|   |-- features.py           # Extraction de features
|   |-- copy_paste.py         # Detection copier-coller
|   |-- plagiarism.py         # Detection plagiat (TF-IDF)
|   |-- scoring.py            # Score final + verdict
|   |-- storage.py            # Stockage predictions
```

```
|   |-- training.py           # Re-entrainement
|-- models/
|   |-- model_v1.pkl          # Modele entraine
|-- storage/
|   |-- predictions.jsonl     # Historique des analyses
|   |-- human_decisions.jsonl # Decisions humaines
|-- scripts/
    |-- validation_complete_huggingface.py  # Tests
```

2.3 Description des Modules

2.3.1 Module de Détection IA (model_ia.py)

Ce module utilise un classifieur **RandomForest** de scikit-learn pour prédire si le code a été généré par une IA.

Features Extraites

1. `mean_line_length` : Longueur moyenne des lignes
2. `std_line_length` : Écart-type de la longueur
3. `indentation_variance` : Variance de l'indentation
4. `comment_ratio` : Ratio de commentaires
5. `max_nesting_depth` : Profondeur d'imbrication
6. `num_branches` : Nombre de if/for/while
7. `avg_identifier_length` : Longueur moyenne des identifiants
8. `num_lines` : Nombre de lignes
9. `empty_line_ratio` : Ratio de lignes vides

2.3.2 Module de Détection Plagiat (plagiarism.py)

Utilise **TF-IDF** (Term Frequency - Inverse Document Frequency) et la **similarité cosinus** pour comparer le code avec des solutions existantes.

$$\text{Score Plagiat} = \max_i \left(\cos(\vec{v}_{\text{code}}, \vec{v}_{\text{solution}_i})\right) \tag{2.1}$$

2.3.3 Module Copier-Coller (copy_paste.py)

Détecte le copier-coller basé sur des règles métier :

— Blocs de plus de 10 lignes insérés en moins de 500ms

— Temps moyen par ligne inférieur à 50ms

2.3.4 Module de Scoring (scoring.py)

Combine les trois scores avec une moyenne pondérée :

$$\text{Score Final} = 0.4 \times \text{score_ia} + 0.3 \times \text{score_plagiat} + 0.3 \times \text{score_copier_coller} \quad (2.2)$$

Score Final	**Verdict**
< 0.35	**OK**
$0.35 - 0.65$	**SUSPECT**
≥ 0.65	**TRICHE PROBABLE**

TABLE 2.1 – Seuils de verdict

Chapitre 3

API REST

3.1 URL de Base

`https://negzaoui-anti-cheat-coding.hf.space`

3.2 Endpoints Disponibles

Méthode	Endpoint	Description
GET	`/health`	Vérifie que l'API est en ligne
GET	`/auto_learn_status`	État de l'apprentissage automatique
POST	`/analyze`	Analyse du code soumis
POST	`/analyze_github`	Analyse d'un repo GitHub
POST	`/human_decision`	Enregistre une décision humaine
POST	`/sync_storage`	Synchronise vers Hugging Face Hub
POST	`/retrain`	Force un ré-entraînement

TABLE 3.1 – Endpoints de l'API

3.3 Endpoint POST /analyze

3.3.1 Requête

Listing 3.1 – Exemple d'appel à /analyze

```
curl -X POST "https://negzaoui-anti-cheat-coding.hf.space/analyze"
    \
  -H "Content-Type: application/json" \
  -d '{"code": "def hello():\n    print(\"world\")"}'
```

3.3.2 Réponse

Listing 3.2 – Réponse de /analyze

```
{
  "score_ia": 0.06,
  "score_plagiat": 0.0,
  "copy_paste_detected": false,
  "score_copy_paste": 0.0,
  "final_score": 0.024,
  "verdict": "OK",
  "submission_id": "sub_1234567890"
}
```

3.4 Endpoint POST /analyze_github

3.4.1 Requête

Listing 3.3 – Exemple d'appel à /analyze_github

```
curl -X POST
   "https://negzaoui-anti-cheat-coding.hf.space/analyze_github" \
  -H "Content-Type: application/json" \
  -d '{"github_url": "https://github.com/user/repo"}'
```

3.4.2 Réponse

Listing 3.4 – Réponse de /analyze_github

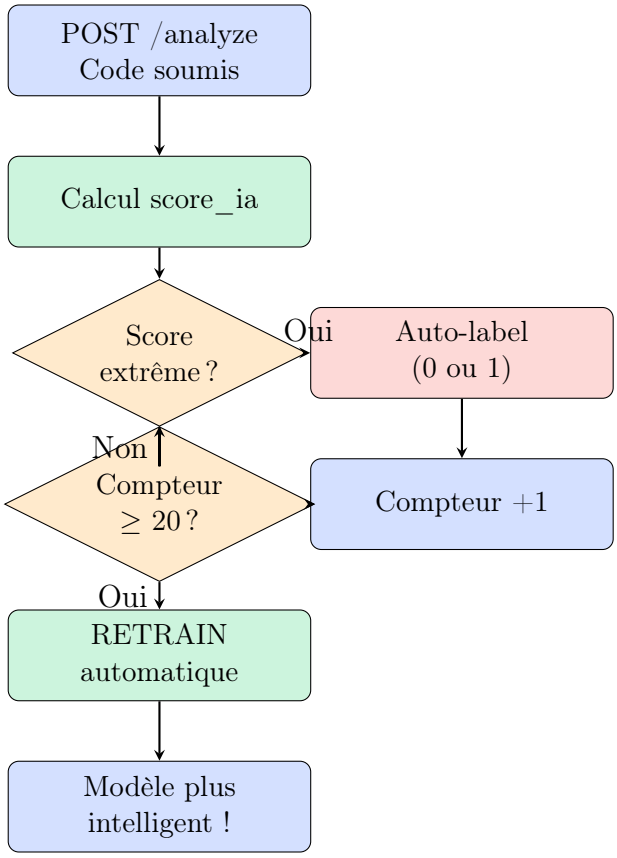
```
{
  "github_url": "https://github.com/user/repo",
  "owner": "user",
  "repo": "repo",
  "files_analyzed": 14,
  "avg_score_ia": 0.86,
  "final_verdict": "TRICHE PROBABLE",
  "details": [
    {"file": "main.py", "score_ia": 0.95, "verdict": "SUSPECT"},
    {"file": "utils.py", "score_ia": 0.77, "verdict": "SUSPECT"}
  ]
}
```

Chapitre 4

Apprentissage Automatique

4.1 Fonctionnement

Le modèle apprend **automatiquement** sans intervention humaine :



4.2 Seuils d'Auto-Labellisation

Score IA	Action	Label
< 0.15	Auto-label	0 (Humain)
$0.15 - 0.85$	Pas d'action	-
> 0.85	Auto-label	1 (IA)

TABLE 4.1 – Seuils d'auto-labellisation

4.3 Configuration

Listing 4.1 – Configuration de l'apprentissage automatique (config.py)

```
# Activer l'apprentissage automatique
AUTO_LEARN_ENABLED = True

# Seuils pour auto-labellisation
AUTO_LABEL_HUMAN_THRESHOLD = 0.15  # score < 0.15 = humain
AUTO_LABEL_IA_THRESHOLD = 0.85     # score > 0.85 = IA

# Re-entrainement apres N decisions
AUTO_RETRAIN_EVERY_N = 20

# Minimum d'echantillons pour retrain
AUTO_RETRAIN_MIN_SAMPLES = 10
```

4.4 Intervention Humaine (Optionnelle)

Si le modèle se trompe, un correcteur peut enregistrer la vraie réponse :

Listing 4.2 – Correction manuelle via /human_decision

```
curl -X POST
   "https://negzaoui-anti-cheat-coding.hf.space/human_decision" \
  -H "Content-Type: application/json" \
  -d '{
    "submission_id": "sub_123",
    "human_label": 0,
    "comment": "C est un humain"
  }'
```

— `human_label: 0` = C'est un humain (OK)

— `human_label: 1` = C'est de la triche (IA)

Chapitre 5

Résultats des Tests

5.1 Validation Complète

Le script `validation_complete_huggingface.py` teste tous les aspects du système.

Resultat : 9/9 tests OK

```
TOTAL: 9 OK / 0 FAIL
```

VALIDATION REUSSIE
Le modele Hugging Face fonctionne !

5.2 Tests Effectués

#	Test	Statut
1	GET /health	✓OK
2	GET /auto_learn_status	✓OK
3	POST /analyze (code humain)	✓OK
4	POST /analyze (code IA)	✓OK
5	POST /analyze (code vide)	✓OK
6	POST /human_decision	✓OK
7	POST /sync_storage	✓OK
8	Test de 12 utilisateurs	✓OK
9	POST /analyze_github	✓OK

TABLE 5.1 – Résumé des tests

5.3 Test des 12 Utilisateurs

Utilisateur	Type	Score IA	Détection
User 1 - Débutant	Humain	0.06	✓
User 2 - Humain erreurs	Humain	0.05	✓
User 3 - Script simple	Humain	0.06	✓
User 4 - Humain expérimenté	Humain	0.37	✓
User 5 - Code brouillon	Humain	0.16	✓
User 6 - Boucle simple	Humain	0.02	✓
User 7 - ChatGPT style	IA	0.93	✓
User 8 - Copilot style	IA	0.67	✓
User 9 - Merge sort IA	IA	0.93	✓
User 10 - Fibonacci IA	IA	0.67	✓
User 11 - Palindrome IA	IA	0.92	✓
User 12 - Classe IA	IA	0.93	✓
		Total :	**12/12 (100%)**

TABLE 5.2 – Résultats du test des 12 utilisateurs

5.4 Test GitHub

Fichier	Score IA	Verdict
config.py	0.95	SUSPECT
src/api.py	0.94	SUSPECT
src/copy_paste.py	0.99	SUSPECT
src/model_ia.py	0.98	SUSPECT
src/features.py	0.93	SUSPECT
src/plagiarism.py	0.94	SUSPECT
src/scoring.py	0.94	SUSPECT
src/storage.py	0.93	SUSPECT
src/training.py	0.93	SUSPECT
Score moyen :	**0.86**	
Verdict final :	TRICHE PROBABLE	

TABLE 5.3 – Analyse du repo GitHub negzaoui16/anti-cheat-coding

Chapitre 6

Code Source Principal

6.1 API REST (api.py)

Listing 6.1 – Extrait de src/api.py

```python
@app.post("/analyze", response_model=AnalyzeResponse)
def analyze_code(request: AnalyzeRequest, background_tasks:
    BackgroundTasks):
    """Analyse le code et retourne les scores + verdict."""
    code = request.code or ""

    # 1. Score IA (modele ML)
    score_ia = predict_ia_score(code)

    # 2. Score plagiat (TF-IDF + cosine)
    score_plagiat = plagiarism_checker.score(code)

    # 3. Copier-coller (regles)
    cp_detected, score_copy_paste = copy_paste_detected(code)

    # 4. Score final et verdict
    result = compute_analysis_result(
        score_ia=score_ia,
        score_plagiat=score_plagiat,
        copy_paste_detected=cp_detected,
        score_copy_paste=score_copy_paste,
    )

    # 5. Apprentissage automatique (arriere-plan)
    background_tasks.add_task(_auto_label_and_maybe_retrain, sid,
        score_ia)

    return AnalyzeResponse(**result)
```

6.2 Extraction de Features (features.py)

Listing 6.2 – Extrait de src/features.py

```
def extract_all_features(code: str) -> dict:
    """Extrait toutes les features du code."""
    lines = code.strip().split("\n")
    mean_len, std_len = extract_line_length_stats(lines)
    indent_var = extract_indentation_variance(lines)
    comment_ratio = extract_comment_ratio(lines)
    max_depth, num_branches, avg_name_len =
       extract_complexity_indicators(code)

    return {
        "mean_line_length": mean_len,
        "std_line_length": std_len,
        "indentation_variance": indent_var,
        "comment_ratio": comment_ratio,
        "max_nesting_depth": max_depth,
        "num_branches": num_branches,
        "avg_identifier_length": avg_name_len,
        "num_lines": len(lines),
        "empty_line_ratio": sum(1 for l in lines if not l.strip())
          / len(lines),
    }
```

6.3 Scoring (scoring.py)

Listing 6.3 – Extrait de src/scoring.py

```
def compute_final_score(score_ia, score_plagiat, score_copy_paste):
    """Score final pondere (entre 0 et 1)."""
    return (
        0.4 * score_ia +
        0.3 * score_plagiat +
        0.3 * score_copy_paste
    )

def get_verdict(final_score):
    """Verdict textuel selon les seuils."""
    if final_score < 0.35:
        return "OK"
    if final_score < 0.65:
        return "SUSPECT"
    return "TRICHE PROBABLE"
```

Chapitre 7

Conclusion

7.1 Récapitulatif

Le système anti-triche est opérationnel

- **9/9 tests** passent avec succès
- **100%** de détection correcte sur 12 utilisateurs
- **Apprentissage automatique** actif (7 auto-labels créés)
- **API déployée** sur Hugging Face Spaces
- **Analyse GitHub** fonctionnelle

7.2 Points Forts

Point Fort	Description
Pas de blocage automatique	Le système donne un score, c'est l'humain qui décide
Apprentissage continu	Le modèle s'améliore avec le temps sans intervention
Léger	Exécutable sur CPU (pas de GPU requis)
API publique	Accessible depuis n'importe quelle application
Open Source	Code disponible sur GitHub

TABLE 7.1 – Points forts du système

7.3 Utilisation Recommandée

1. **Intégrer l'API** dans votre plateforme de coding challenges

2. **Appeler POST /analyze** pour les soumissions de code
3. **Appeler POST /analyze_github** pour les projets GitHub
4. **Laisser le modèle apprendre** automatiquement
5. **Optionnel :** Corriger les erreurs via POST /human_decision

Le modèle est prêt pour la production !